

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
“ВЫСШАЯ ШКОЛА ЭКОНОМИКИ”**

Факультет компьютерных наук
Образовательная программа “Программная инженерия”

СОГЛАСОВАНО

Научный руководитель, инженер-разработчик, приглашённый преподаватель департамента программной инженерии

_____ / Е.А. Караваева /
“ ” _____ 2026 г.

УТВЕРЖДАЮ

Академический руководитель образовательной программы «Программная инженерия», старший преподаватель департамента программной инженерии

_____ / Н.А. Павлов /
“ ” _____ 2026 г.

**ВЕБ-ПРИЛОЖЕНИЕ ДЛЯ ОНЛАЙН-ОБУЧЕНИЯ
С ПОДДЕРЖКОЙ ВИДЕОСВЯЗИ
И ИИ-ПОМОЩНИКА.
СЕРВЕРНАЯ ЧАСТЬ 1**

Программа и методика испытаний

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.12.17-01 51 01-3-ЛУ

Исполнитель:
Студент группы БПИ244

_____ / С.М. Павлычев /
“ ” _____ 2026 г.

Москва 2026

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подл.	

УТВЕРЖДЁН
RU.17701729.12.17-01 51 01-3-ЛУ
11.05.2026

**ВЕБ-ПРИЛОЖЕНИЕ ДЛЯ ОНЛАЙН-ОБУЧЕНИЯ
С ПОДДЕРЖКОЙ ВИДЕОСВЯЗИ
И ИИ-ПОМОЩНИКА.
СЕРВЕРНАЯ ЧАСТЬ 1**

Программа и методика испытаний

RU.17701729.12.17-01 51 01-3

Листов 33

Инов.№ подл.	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

АННОТАЦИЯ

Программа и методика испытаний — это документ, в котором содержится информация о программном продукте, а также полное описание приёмочных испытаний для данного программного продукта.

Настоящая Программа и методика испытаний для «Веб-приложения для онлайн-обучения с поддержкой видеосвязи и ИИ-помощника. Серверная часть 1» содержит следующие разделы: «Объект испытаний», «Цель испытаний», «Требования к программе», «Требования к программной документации», «Средства и порядок испытаний», «Методы испытаний», «Приложения».

В разделе «Объект испытаний» указано наименование, краткая характеристика и назначение программы.

В разделе «Цель испытаний» указана цель проведения испытаний. Раздел «Требования к программе» содержит основные требования к программе, которые подлежат проверке во время испытаний (требования к функционалу и интерфейсу).

Раздел «Требования к программной документации» содержит состав программной документации, которая представляется на испытания.

Раздел «Средства и порядок испытаний» содержит информацию о технических и программных средствах, которые следует использовать во время испытаний, а также порядок этих испытаний.

Раздел «Методы испытаний» содержит информацию об используемых методах испытаний.

Настоящий документ разработан в соответствии с требованиями:

1. ГОСТ 19.101-77 [1]: Виды программ и программных документов.
2. ГОСТ 19.102-77 [2]: Стадии разработки.
3. ГОСТ 19.103-77 [3]: Обозначения программ и программных документов.
4. ГОСТ 19.104-78 [4]: Основные надписи.
5. ГОСТ 19.105-78 [5]: Общие требования к программным документам.
6. ГОСТ 19.106-78 [6]: Требования к программным документам, выполненным печатным способом.
7. ГОСТ 19.301-79 [7]: Программа и методика испытаний. Требования к содержанию и оформлению.

Изменения к настоящей программе и методике испытаний должны быть оформлены согласно ГОСТ 19.603-78 [8] и ГОСТ 19.604-78 [9].

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

СОДЕРЖАНИЕ

АННОТАЦИЯ	2
1. ОБЪЕКТ ИСПЫТАНИЙ	5
1.1. Наименование программы	5
1.2. Область применения	5
1.3. Состав испытываемых компонентов	5
2. ЦЕЛЬ ИСПЫТАНИЙ	7
3. ТРЕБОВАНИЯ К ПРОГРАММЕ	8
3.1. Функциональные требования	8
3.1.1. Подсистема содержимого курсов	8
3.1.2. Подсистема домашних заданий и попыток	8
3.1.3. Подсистема платежей	9
3.1.4. Подсистема уведомлений	9
3.1.5. Подсистема ИИ-помощника	9
3.1.6. Подсистема модераторской панели	10
3.1.7. Подсистема заявок на специальные курсы	10
3.1.8. Подсистема медиа-ассетов	10
3.2. Нефункциональные требования	11
3.2.1. Требования к надёжности	11
3.2.2. Требования к безопасности	11
3.2.3. Требования к временным характеристикам	11
4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ	12
5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ	13
5.1. Технические средства	13
5.2. Программные средства	13
5.3. Порядок проведения испытаний	13
6. МЕТОДЫ ИСПЫТАНИЙ	15
6.1. Статический анализ кода (этап 1)	15
6.1.1. Метод проведения	15
6.2. Дымовая проверка (этап 2)	15
6.2.1. Метод проведения	15
6.3. Модульное и компонентное тестирование (этап 3)	15
6.3.1. Метод проведения	15
6.3.2. Тестирование подсистемы каталога курсов	16
6.3.3. Тестирование подсистемы домашних заданий (apps.homeworks)	17
6.3.4. Тестирование подсистемы платежей	18
6.3.5. Тестирование подсистемы уведомлений	18
6.3.6. Тестирование ИИ-помощника	19
6.3.7. Тестирование модераторской панели	20
6.3.8. Тестирование заявок на специальные курсы	20

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

6.4. Сквозное тестирование (этап 4)	21
6.4.1. Метод проведения	21
6.4.2. Сценарий 1: публикация курса и доступ зачисленного студента к материалам	21
6.4.3. Сценарий 2: полный цикл домашнего задания с уведомлением	22
6.4.4. Сценарий 3: заявка на специальный курс – зачисление и доступ	22
6.4.5. Сценарий 4: маршрутизация уведомлений через диспетчер	22
6.4.6. Плановые показатели покрытия кода	23
6.5. Ручная приёмка безопасности (этап 5)	23
6.5.1. Метод проведения	23
6.5.2. Перечень проверок	23
6.5.3. Защита от утечки технических деталей	24
ПРИЛОЖЕНИЕ 1 – СВОДНАЯ ТАБЛИЦА ТЕСТ-СЦЕНАРИЕВ	25
ПРИЛОЖЕНИЕ 2 – ПРОТОКОЛ ИЗМЕРЕНИЯ ПОКРЫТИЯ КОДА	27
ПРИЛОЖЕНИЕ 3 – СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ	28
ПРИЛОЖЕНИЕ 4 – ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ТЕРМИНОВ	31

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1. ОБЪЕКТ ИСПЫТАНИЙ

1.1. Наименование программы

Объектом испытаний является серверная часть 1 программного изделия «Веб-приложение для онлайн-обучения с поддержкой видеосвязи и ИИ-помощника» (краткое наименование – «Профессия»).

Обозначение программы по ГОСТ 19.103-77: **RU.17701729.12.17-01**.

Вид программного изделия по ГОСТ 19.101-77: компонент комплекса программ – серверная подсистема, реализующая бизнес-логику и REST API в составе функциональных модулей платформы.

1.2. Область применения

Серверная часть 1 применяется в составе образовательной платформы «Профессия», предназначенной для организации дистанционного обучения в коммерческом и корпоративном формате.

Серверная часть 1 ориентирована на организаторов образовательных платформ, преподавателей и студентов и применяется для обеспечения следующих видов деятельности:

- публикация образовательных курсов и управление их содержимым организациями и преподавателями;
- приобретение студентами доступа к курсам через платёжный шлюз либо по специальным заявкам;
- выдача, приём и оценивание учебных заданий в процессе прохождения курсов;
- доставка уведомлений участникам платформы об учебных событиях (оценки, дедлайны, обновления курса);
- поддержка студентов в ходе обучения посредством ИИ-помощника, осведомлённого о материалах курса;
- администрирование платформы: управление составом преподавателей, публикация и снятие курсов.

Взаимодействие пользователей с системой осуществляется посредством клиентского веб-приложения (React 19, TypeScript), которое обращается к серверной части 1 по протоколам HTTP (REST API) и WebSocket.

1.3. Состав испытуемых компонентов

Таблица 1.1 – Состав испытуемых компонентов серверной части 1

Компонент	Назначение	Основные сущности
courses	Каталог курсов, иерархия секций, уроков и ДЗ, кеш Redis, публичный лендинг	Course, Section, Lesson, Homework, Question, Task

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Компонент	Назначение	Основные сущности
homeworks	Попытки студентов, автоматическая и ручная проверка, уведомление об оценке	Attempt, QuestionAnswer, TaskAnswer, TaskReview
payments	Платёжный цикл через сервис MockYooKassaService, Celery-обработка платежей	Payment, PaymentItem, CourseEnrollment
notifications	SSE-поток событий через RabbitMQ, история уведомлений, email и SMS	Notification, NotificationDispatcher
ai_chat_bot	WebSocket-помощник на базе Yandex GPT, управление чатами и историей	Session, Chat, Message
admin_panel	Управление преподавателями, одноразовые токены-приглашения, публикация	Invitation
applications	Заявки студентов на специальные курсы без оплаты	CourseApplication
core	Медиа-ассеты: S3-загрузка, отслеживание ссылок, очистка устаревших файлов	MediaAsset, AssetUsage

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. ЦЕЛЬ ИСПЫТАНИЙ

Целью испытаний является проверка соответствия серверной части 1 Платформы требованиям технического задания (RU.17701729.12.17-01 ТЗ 01-3) в части функциональности, надёжности и безопасности в зоне ответственности исполнителя С.М. Павлычева.

В ходе испытаний проверяются:

1. корректность реализации функциональных требований по всем подсистемам зоны ответственности: каталог курсов, домашние задания, платежи, уведомления, ИИ-помощник, административная панель, заявки, загрузка файлов;
2. правильность работы ролевой модели доступа (RBAC): разграничение прав student, teacher, moderator при обращении к защищённым REST API и WebSocket-эндпоинтам;
3. корректность бизнес-логики домашних заданий: атомарное создание попытки, автопроверка вопросов с вариантами ответа, ручная оценка задач преподавателем, уведомление студента;
4. корректность платёжного цикла: создание платежа со статусом pending, обработка webhook-уведомления, создание записи CourseEnrollment (зачисление на курс);
5. корректность доставки SSE-уведомлений: подписка через RabbitMQ topic exchange, heartbeat каждые 30 секунд, до 5 повторных попыток подключения к брокеру;
6. корректность WebSocket-протокола ИИ-помощника: управление чатами, запрос истории, стриминг ответов Yandex GPT токеном за токеном;
7. корректность работы механизма медиа-ассетов: создание presigned URL, смена статуса ассета с pending на ready, периодические Celery-задачи очистки;
8. надёжность Celery-задач: повторные попытки при сбоях с экспоненциальной задержкой, корректное завершение при превышении числа попыток;
9. защита от утечки технических деталей ошибок в HTTP-ответах клиенту;
10. соответствие временным характеристикам: медианное время REST API $\leq 1,5$ с; время до первого токена ИИ-помощника ≤ 3 с; доставка SSE-уведомления ≤ 3 с.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. ТРЕБОВАНИЯ К ПРОГРАММЕ

Настоящая программа и методика испытаний (далее – ПМИ) определяет объём и состав испытаний в соответствии с требованиями технического задания исполнителя С.М. Павлычева (RU.17701729.12.17-01 ТЗ 01-3, раздел 4) и руководства программиста (RU.17701729.12.17-01 33 01-3).

3.1. Функциональные требования

3.1.1. Подсистема содержимого курсов

1. Система предоставляет публичный каталог опубликованных курсов через GET /api/landing/courses/ с кешированием в Redis (TTL $\geq$ 600 с).
2. Модератор и преподаватель – автор курса – располагают полным доступом к созданию, редактированию и удалению секций, уроков, домашних заданий, вопросов и задач; пользователь с ролью student осуществляет исключительно чтение опубликованных материалов.
3. Порядковые номера секций и уроков (section_number, lesson_number) инкрементируются автоматически в пределах родительского элемента.
4. При сохранении урока временные ссылки local:
5. Студент, имеющий активную запись CourseEnrollment (поле is_active возвращает True), получает доступ к материалам урока; при отсутствии записи о зачислении либо при истечении срока действия поля access_expires_at сервер возвращает HTTP 403.
6. При изменении домашнего задания система публикует уведомления и планирует Celery-напоминания за 24 часа и за 1 час до дедлайна.
7. Эндпоинт пользовательского сервиса: GET /api/my-content/ (объединённое содержимое уроков пользователя). Эндпоинты GET /api/my-courses/ и GET /api/my-schedule/ реализуются в зоне ответственности исполнителя П.Д. Комковой; исполнитель обеспечивает предоставление данных CourseEnrollment и дедлайнов домашних заданий.

3.1.2. Подсистема домашних заданий и попыток

1. Сервис AttemptService.get_or_create_draft атомарно создаёт или возвращает черновик попытки (ограничение unique_together).
2. Сервис AttemptService.submit выполняет валидацию входных данных: проверяет соответствие attempt_id, допустимость статуса попытки (только draft) и готовность приложенных медиа-ассетов; после успешной валидации запускает AutocheckService для вопросов с заранее определённым правильным ответом и вычисляет итоговый балл попытки.
3. ReviewService поддерживает выставление баллов и комментариев за задачи с развёрнутым ответом (не более 1500 знаков); уведомляет студента через NotificationDispatcher.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4. Эндпоинты подсистемы: GET/POST /api/courses/:slug/homeworks/:hw_slug/attempt/; POST .../attempt/submit/; GET /api/courses/:slug/attempt/:attempt_id/; POST/PATCH .../review/; GET /api/my-homeworks/.

3.1.3. Подсистема платежей

1. POST /api/carts/pay/ создаёт запись Payment со статусом pending и полями mock_yookassa_id, mock_payment_url, total_sum, а также связанные записи PaymentItem (payment × course × price, уникальность пары); в ответе возвращается URL платёжной формы подменного сервиса MockYooKassaService с success-rate ≈ 80 %.
2. Celery-задача process_payment_task обрабатывает платёж асинхронно с числом повторов до 5 и задержкой 15 секунд; при успехе переводит Payment в статус success, при исчерпании попыток – в статус failed.
3. При успешной оплате создаётся запись CourseEnrollment с полем source = «payment» и access_expires_at, рассчитанным согласно политике курса.
4. Эндпоинты просмотра истории: GET /api/payments/, GET /api/payments/:payment_id/.

3.1.4. Подсистема уведомлений

1. Внутриплатформенные уведомления доступны через эндпоинт GET /api/notifications/ с курсорной пагинацией; пометка всех уведомлений прочитанными выполняется через POST /api/notifications/read-all/.
2. SSE-поток GET /api/notifications/sse/?token=...: события из RabbitMQ topic exchange notifications с routing-ключами user.id, course.id, system.all, webinar.id; heartbeat каждые 30 секунд; до 5 повторных попыток подключения к RabbitMQ.
3. Email через Django mail backend (SMTP) и SMS через Notificore.
4. Диспетчер событий NotificationDispatcher маршрутизирует следующие типы событий во все настроенные каналы доставки: CourseUpdatedEvent, AuthorActionEvent, NewHomeworkEvent, DeadlineChangedEvent. События вебинарного расписания (WebinarScheduledEvent, WebinarStartedEvent) публикуются подсистемой вебинаров в зоне ответственности П.Д. Комковой и обрабатываются тем же диспетчером.

3.1.5. Подсистема ИИ-помощника

1. WebSocket-эндпоинт ws/api/courses/:course_slug/ai/chat/ (ChatConsumer) с обязательной JWT-аутентификацией.
2. Поддерживаемые типы входящих сообщений: start new chat, delete chat, get history, send message (поле content.text, не более 1000 символов).
3. Исходящие сообщения: connected (список чатов сессии при подключении), chat created, chat deleted, history received, starting answer, streaming response (поток токенов), finishing answer, error.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4. YandexChatAIService обеспечивает стриминг ответов с семафором ограничения параллелизма и ведёт историю чата.
5. YandexKnowledgeAIService загружает содержимое курса в Yandex Vector Store методом RAG; синхронизация инициируется Celery-задачей `synchronize_course_context`.
6. Модерация содержания: ИИ-помощник отклоняет запросы, содержащие нецензурную лексику, оскорбительные высказывания, а также вопросы, выходящие за пределы тематики конкретного курса или нарушающие требования действующего законодательства Российской Федерации.

3.1.6. Подсистема модераторской панели

1. POST `/api/admin-panel/invites/send/` создаёт токен приглашения (64 знака, TTL 3 суток) и отправляет письмо преподавателю. Доступно только moderator.
2. Эндпоинты: GET `/api/admin-panel/teachers/`, GET `/api/admin-panel/invites/`, POST `/api/admin-panel/invites/validate/`, POST `/api/admin-panel/invites/register/`.
3. POST `/api/admin-panel/courses/:slug/add-author/`, DELETE `/api/admin-panel/courses/:slug/remove-author/`.
4. POST `/api/admin-panel/courses/:slug/publish/`, POST `/api/admin-panel/courses/:slug/unpublish/`.
5. По истечении установленного срока действия приглашение автоматически переводится системой в статус `expired`, исключая возможность его повторного использования.

3.1.7. Подсистема заявок на специальные курсы

1. Студент подаёт заявку через POST `/api/courses/:slug/applications/apply/`; уникальность по паре `user × course`.
2. Преподаватель-автор и модератор одобряют (`approved`) или отклоняют (`rejected`) заявку. Обратные переходы запрещены.
3. При одобрении создаётся запись `CourseEnrollment` с полем `source = «application»` без привязки к платежу; студенту доставляется внутриплатформенное уведомление об одобрении заявки.

3.1.8. Подсистема медиа-ассетов

1. POST `/api/uploads/initiate/` создаёт запись `MediaAsset` со статусом `pending` и возвращает `presigned URL` для прямой загрузки файла в Yandex S3.
2. GET `/api/uploads/:asset_id/status/` возвращает статус ассета.
3. Периодические фоновые задачи: `sweep_pending_assets` (каждые 600 с, TTL 24 ч)– удаление зависших `pending`-ассетов; `sweep_orphaned_assets` (каждые 3600 с, `grace` 7 дней)– физическое удаление неиспользуемых файлов; `poll_s3_upload_events` (каждые 30 с)– опрос SQS-очереди событий S3.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.2. Нефункциональные требования

3.2.1. Требования к надёжности

1. Фоновые задачи Celery – отправка уведомлений, обработка платежей, синхронизация контекста RAG, физическое удаление ассетов, рассылка напоминаний о дедлайнах – при возникновении сбоя повторяются с экспоненциально возрастающей задержкой; после исчерпания допустимого числа попыток задача фиксируется в журнале для последующего ручного анализа.
2. При обрыве SSE-соединения серверная часть корректно завершает потоковую передачу; при повторном подключении клиента доставка возобновляется с учётом актуального состояния очереди брокера.
3. Отказ одного из внешних сервисов (Yandex GPT, Kinescope, SMTP, Notificore, RabbitMQ, S3) не приводит к деградации или полной недоступности серверной части в целом: соответствующие эндпоинты возвращают клиенту интерпретируемое сообщение об ошибке.

3.2.2. Требования к безопасности

1. Обращения к защищённым ресурсам без действительного токена аутентификации отклоняются сервером с возвратом HTTP 401.
2. Запросы, выполненные от имени пользователя с недостаточными ролевыми полномочиями, отклоняются с возвратом HTTP 403.
3. Внутренние технические детали ошибок – текст SQL-запроса, трассировка стека вызовов Python, диагностические сообщения драйвера базы данных – ни при каких обстоятельствах не включаются в HTTP-ответ, возвращаемый клиенту.

3.2.3. Требования к временным характеристикам

1. Медианное время ответа REST API (p50): $\leq 1,5$ с; p95 ≤ 2 с.
2. Время до первого токена ИИ-помощника через WebSocket: ≤ 3 с от отправки вопроса.
3. Время доставки SSE-уведомления: ≤ 3 с от момента публикации события в RabbitMQ.
4. Время обработки платежа (включая до 5 повторов): ≤ 30 с.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

На испытания предъявляется следующий состав программной документации:

1. Техническое задание – RU.17701729.12.17-01 ТЗ 01-3 (ГОСТ 19.201-78).
2. Текст программы – RU.17701729.12.17-01 12 01-3 (ГОСТ 19.401-78).
3. Руководство программиста – RU.17701729.12.17-01 33 01-3 (ГОСТ 19.504-79).
4. Программа и методика испытаний – RU.17701729.12.17-01 51 01-3 (настоящий документ).

Документация оформлена в соответствии с ГОСТ 19.106-78.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ

5.1. Технические средства

Испытания проводятся на следующем оборудовании:

Таблица 5.1 – Технические средства проведения испытаний

Параметр	Значение
Процессор	Apple M1 Pro, 10 ядер / 10 потоков
Оперативная память	16 ГБ LPDDR5
Постоянное хранилище	SSD NVMe 512 ГБ
Операционная система	macOS Sequoia 15.x (Darwin 24.6.0)
Docker Engine	Docker Desktop for Mac 4.x
Python	версия 3.12.x
Node.js	версия 20 LTS

5.2. Программные средства

Таблица 5.2 – Программные средства проведения испытаний

Средство	Назначение
django.test.TestCase	Основной фреймворк модульного тестирования Django
APIClient (DRF)	HTTP-клиент для тестирования REST API через Django REST Framework
WebsocketCommunicator	Тестирование Django Channels WebSocket-консьюмеров
unittest.mock.patch	Изоляция внешних зависимостей (Yandex GPT, S3, Notificore, MockYooKassaService)
coverage.py	Измерение покрытия кода тестами с генерацией HTML-отчёта
ruff	Статический анализ Python-кода; выявление ошибок до деплоя
Docker Compose	Оркестрация контейнеров PostgreSQL, Redis, RabbitMQ для интеграционных тестов
Postman / cURL	Ручная проверка REST API, SSE-потока и WebSocket

5.3. Порядок проведения испытаний

Испытания проводятся в пять последовательных этапов, соответствующих классической пирамиде тестирования. Каждый этап является обязательным; переход к следующему допускается исключительно после документально подтверждённого успешного завершения предыдущего.

1. **Первый этап – статический анализ кода.** Выполняется первым, поскольку не требует запуска приложения и позволяет обнаружить грубые ошибки стиля и типизации до начала тестирования. Запускается инструмент `ruff check apps/`. Критерий: код выхода 0, отсутствие предупреждений уровня `error`.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. **Второй этап – дымовая проверка (smoke).** Серверная часть запускается через docker compose up -d backend. Проверяется, что приложение успешно стартует и отвечает на запросы к диагностическим эндпоинтам GET /health/live/ и GET /health/ready/. Критерий: оба эндпоинта возвращают HTTP 200. Провал дымовой проверки означает фундаментальную неработоспособность стека и исключает переход к следующим этапам.
3. **Третий этап – модульное и компонентное тестирование.** Охватывает три уровня: изолированные unit-тесты доменных сущностей и сервисов (без БД, без HTTP), HTTP-тесты отдельных эндпоинтов через APIClient (реальная тестовая БД, внешние сервисы замещены unittest.mock.patch) и pipeline-тесты, проверяющие связность нескольких компонентов (диспетчер уведомлений, Celery-задачи в режиме немедленного выполнения). Покрытие кода измеряется попутно командой coverage run и не выносится в отдельный этап. Критерий: все тест-функции со статусом PASS, плановые показатели покрытия достигнуты (Таблица 6.7).
4. **Четвёртый этап – сквозное тестирование.** Выполняются многошаговые сценарии, охватывающие полный жизненный цикл операции с участием нескольких ролей: от первого HTTP-запроса до проверки побочных эффектов – состояния объектов в базе данных, содержимого mail.outbox и перехваченных SSE-событий. Внешние сервисы замещаются unittest.mock.patch. Критерий: все сценарии завершились со статусом PASS.
5. **Пятый этап – ручная приёмка безопасности.** Выполняется вручную через Postman / cURL против запущенного стека. Проверяются сценарии, которые сложно полностью автоматизировать: истёкший JWT-токен, токен с неверной подписью, повышение привилегий, утечка технических деталей в теле HTTP-ответа при внутренней ошибке. Критерий: все пункты ручного чек-листа (раздел 6, подраздел «Ручная приёмка безопасности») отмечены как выполненные.

Испытания в целом признаются пройденными при условии успешного завершения всех пяти этапов.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

6. МЕТОДЫ ИСПЫТАНИЙ

6.1. Статический анализ кода (этап 1)

6.1.1. Метод проведения

Статический анализ выполняется инструментом `ruff` до запуска любых тестов и не требует работающей базы данных или запущенного сервера.

Листинг 1.1 – Запуск статического анализа серверной части 1

```
cd backend  
ruff check apps/
```

Критерий годности: код выхода 0, отсутствие предупреждений уровня `error`.

6.2. Дымовая проверка (этап 2)

6.2.1. Метод проведения

Дымовая проверка устанавливает базовую работоспособность стека: приложение должно успешно запускаться и отвечать на диагностические запросы. Проверка выполняется против работающего контейнера серверной части.

Листинг 2.1 – Запуск стека и дымовая проверка

```
docker compose up -d backend  
curl -f http://localhost:9000/health/live/  
curl -f http://localhost:9000/health/ready/
```

Критерий годности: оба запроса завершаются с HTTP 200. Провал дымовой проверки прекращает испытания до устранения дефекта.

6.3. Модульное и компонентное тестирование (этап 3)

6.3.1. Метод проведения

Тестирование охватывает три подуровня, запускаемых единой командой:

1. **Unit-тесты** – проверяют доменные сущности и сервисный слой в полной изоляции: база данных не используется, HTTP-слой не задействован. Примеры: `test_models.py`, `test_attempt_service.py`, `test_autocheck_service.py`, `test_ws_request_parse.py`.
2. **HTTP-тесты** – проверяют отдельные эндпоинты через `APIClient` против реальной тестовой базы данных; все внешние сервисы (`Yandex GPT`, `S3`, `Notificore`, `MockYooKassaService`, `email`) замещаются `unittest.mock.patch`. Примеры: `test_views.py`, `test_homework_attempt_submit_and_review_http.py`, `test_moderator_course_authors_publish_http.py`.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. **Pipeline-тесты** – проверяют связность нескольких компонентов в рамках одной транзакции: диспетчер уведомлений, Celery-задачи в режиме синхронного выполнения через `side_effect`. Примеры: `test_homework_reviewed_dispatch_pipeline.py`, `test_notification_tasks.py`.

Покрывание кода (*code coverage*) – метрика, показывающая долю исходного кода, выполненную хотя бы одним тестом. В настоящем документе покрытие измеряется инструментом `coverage.py`, который перехватывает выполнение строк кода во время прогона тестов и формирует отчёт о том, какие строки, ветви и функции были задействованы. Покрытие является вспомогательным показателем качества тестирования и измеряется попутно с запуском тестов – без выделения в самостоятельный этап испытаний.

Листинг 3.1 – Запуск модульного тестирования с измерением покрытия

```
cd backend
coverage run --source=apps manage.py test apps -v 2
coverage report -m
```

6.3.2. Тестирование подсистемы каталога курсов

1. Проверяемые сценарии:

- `test_list_courses_returns_only_published` – каталог возвращает только опубликованные курсы (HTTP 200);
- `test_list_courses_uses_cache` – повторный запрос обслуживается из Redis без SQL-запроса к БД;
- `test_cache_invalidated_on_course_update` – инвалидация кеша при изменении курса;
- `test_teacher_author_can_create_section` – автор курса создаёт секцию (HTTP 201);
- `test_student_cannot_create_section` – студент не создаёт секцию (HTTP 403);
- `test_non_author_teacher_cannot_edit` – преподаватель без авторства не редактирует чужой курс (HTTP 403);
- `test_section_number_auto_increment` – `section_number` инкрементируется в рамках курса;
- `test_lesson_local_links_replaced` – при сохранении урока ссылки `local`;
- `test_enrolled_student_accesses_lesson` – купивший студент получает урок (HTTP 200);
- `test_not_enrolled_student_blocked` – студент без записи получает HTTP 403;
- `test_expired_enrollment_blocked` – студент с истёкшим `access_expires_at` получает HTTP 403;
- `test_homework_signals_schedule_reminders` – при изменении ДЗ планируются Celery-напоминания;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- test_my_courses_returns_purchased – GET /api/my-courses/ возвращает купленные курсы.

Таблица 6.1 – Результаты модульного тестирования -- «courses»

Файл тестов	Тестов	Результат	Время, с
test_models.py	12	PASS	0,480
test_views.py	18	PASS	1,120
test_serializers.py	8	PASS	0,210
test_signals.py	5	PASS	0,340
test_decorators.py	4	PASS	0,130
test_lesson_content.py	6	PASS	0,280
test_courses_flow_e2e.py	7	PASS	1,540
Итого	60	PASS	4,100

6.3.3. Тестирование подсистемы домашних заданий (apps.homeworks)

1. Проверяемые сценарии:

- test_get_or_create_draft_atomicity – атомарное создание черновика попытки; повторный вызов возвращает существующую запись;
- test_submit_validates_attempt_id – submit отклоняет несовпадающий attempt_id (HTTP 400);
- test_submit_rejects_submitted_attempt – повторная отправка уже отправленной попытки отклоняется (HTTP 409);
- test_autocheck_scores_questions – AutocheckService правильно вычисляет баллы за вопросы с вариантами;
- test_autocheck_ignores_task_type – задачи с произвольным ответом не автопроверяются;
- test_review_sets_grade_and_notifies – ReviewService выставляет балл и публикует уведомление;
- test_review_comment_max_length – комментарий длиннее 1500 знаков отклоняется;
- test_teacher_reviews_attempt_http – преподаватель выставляет оценку через HTTP (HTTP 200);
- test_student_sees_own_attempts – студент получает список своих попыток (HTTP 200);
- test_teacher_sees_attempts_of_own_courses – преподаватель видит попытки по своим курсам;
- test_student_cannot_see_others_attempts – студент не видит попытки другого студента (HTTP 403).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Таблица 6.2 – Результаты модульного тестирования -- «homeworks»

Файл тестов	Тестов	Результат	Время, с
test_attempt_service.py	8	PASS	0,320
test_autocheck_service.py	6	PASS	0,180
test_review_service.py	5	PASS	0,160
test_errors.py	4	PASS	0,090
test_homework_attempt_submit_and_review_http.py	10	PASS	0,850
test_homework_review_student_notification_pipeline.py	6	PASS	0,720
test_integration.py	4	PASS	1,100
test_e2e.py	5	PASS	1,440
Итого	48	PASS	4,860

6.3.4. Тестирование подсистемы платежей

1. Проверяемые сценарии:

- test_initiate_payment_creates_pending – инициирование оплаты создаёт Payment со статусом pending и возвращает URL (HTTP 200);
- test_mock_service_success_rate – успешность платежей подменного сервиса MockYooKassaService составляет $\approx 80\%$;
- test_webhook_success_creates_enrollment – успешный webhook создаёт запись CourseEnrollment;
- test_webhook_failed_sets_terminal_status – неуспешный webhook переводит Payment в терминальный статус;
- test_celery_retries_on_failure – при сбое задача process_payment_task повторяется до 5 раз;
- test_payment_history_accessible – GET /api/payments/ возвращает историю (HTTP 200);
- test_student_cannot_see_others_payments – студент не видит платежи другого студента (HTTP 403).

6.3.5. Тестирование подсистемы уведомлений

1. Проверяемые сценарии:

- test_sse_authenticated_returns_stream – аутентифицированный клиент получает SSE-поток (HTTP 200, Content-Type: text/event-stream);
- test_sse_unauthenticated_rejected – запрос без токена отклоняется;
- test_notification_list_paginated – GET /api/notifications/ возвращает список с курсорной пагинацией;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- test_read_all_marks_notifications – POST /api/notifications/read-all/ помечает все уведомления прочитанными;
- test_dispatcher_routes_homework_event – NotificationDispatcher публикует событие NewHomeworkEvent;
- test_dispatcher_routes_deadline_changed – публикация события DeadlineChangedEvent;
- test_celery_notification_retry_on_failure – при сбое задача уведомления повторяется.

Таблица 6.3 – Результаты модульного тестирования -- «notifications»

Файл тестов	Тестов	Результат	Время, с
test_dispatcher_service.py	6	PASS	0,240
test_homework_reviewed_dispatch_pipeline.py	4	PASS	0,530
test_notification_tasks.py	5	PASS	0,310
test_notifications_api_http.py	7	PASS	0,620
Итого	22	PASS	1,700

6.3.6. Тестирование ИИ-помощника

1. Проверяемые сценарии:

- test_connect_without_auth_closed_4003 – подключение без токена закрывается кодом 4003;
- test_connect_sends_connected_with_chats – при подключении сервер отправляет фрейм connected со списком чатов;
- test_start_new_chat_response – start new chat создаёт чат и возвращает chat created;
- test_send_message_streams_response – send message инициирует поток streaming response-фреймов;
- test_get_history_returns_messages – get history возвращает историю чата;
- test_delete_chat – delete chat удаляет чат и возвращает chat deleted;
- test_unsupported_type_returns_error – неизвестный тип сообщения возвращает error;
- test_message_content_max_length – сообщение длиннее 1000 символов отклоняется;
- test_ws_request_parse_valid – корректный фрейм успешно разбирается;
- test_ws_request_parse_missing_chat_id – фрейм без обязательного поля отклоняется.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Таблица 6.4 – Результаты модульного тестирования -- «ai_chat_bot»

Файл тестов	Тестов	Результат	Время, с
test_chat_consumer_websocket.py	8	PASS	1,240
test_ws_request_parse.py	6	PASS	0,110
test_yandex_chat_ai_service_orm.py	4	PASS	0,350
Итого	18	PASS	1,700

6.3.7. Тестирование модераторской панели

1. Проверяемые сценарии:

- test_invitation_created_with_ttl – токен приглашения создаётся с expires_at через 3 суток;
- test_invitation_expires_automatically – по истечении срока статус автоматически меняется на expired;
- test_send_invite_moderator_only – отправка приглашения доступна только moderator;
- test_validate_invite_token_success – валидный токен успешно проверяется;
- test_validate_invite_token_expired – истёкший токен отклоняется (HTTP 400);
- test_register_via_invite_sets_teacher_role – регистрация по инвайту устанавливает роль teacher;
- test_add_author_to_course – добавление автора к курсу (HTTP 200);
- test_remove_author_from_course – удаление автора из состава авторов курса (HTTP 200);
- test_publish_course_by_moderator – публикация курса (HTTP 200);
- test_unpublish_course_by_moderator – снятие с публикации (HTTP 200).

Таблица 6.5 – Результаты модульного тестирования -- «admin_panel»

Файл тестов	Тестов	Результат	Время, с
test_invitation_model.py	5	PASS	0,190
test_serializers.py	4	PASS	0,120
test_moderator_course_authors_publish_http.py	8	PASS	0,650
test_moderator_teacher_directory_http.py	4	PASS	0,310
test_public_teacher_invite_token_validate_http.py	4	PASS	0,280
test_teacher_registration_via_invite_http.py	5	PASS	0,420
Итого	30	PASS	1,970

6.3.8. Тестирование заявок на специальные курсы

1. Проверяемые сценарии:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- test_student_can_apply_to_special_course – студент подаёт заявку (HTTP 201);
- test_duplicate_application_rejected – повторная заявка отклоняется (HTTP 409);
- test_teacher_cannot_apply – преподаватель не может подать заявку (HTTP 403);
- test_approve_creates_enrollment – одобрение заявки создаёт CourseEnrollment с source = «application» и уведомляет студента;
- test_reject_does_not_grant_access – отклонение заявки не создаёт запись CourseEnrollment;
- test_reverse_transition_forbidden – возврат из approved в pending запрещён;
- test_special_course_not_visible_in_public_catalog – специальный курс скрыт в публичном каталоге.

Таблица 6.6 – Результаты модульного тестирования -- «applications»

Файл тестов	Тестов	Результат	Время, с
test_application_submit_and_review_http.py	8	PASS	0,720
test_application_approve_enrollment_pipeline.py	5	PASS	0,490
test_special_course_visibility.py	4	PASS	0,330
Итого	17	PASS	1,540

6.4. Сквозное тестирование (этап 4)

6.4.1. Метод проведения

Сквозные тесты проверяют полный жизненный цикл операции посредством последовательных HTTP-запросов от имени нескольких ролей. По завершении каждого шага фиксируется не только HTTP-статус ответа, но и совокупность побочных эффектов: состояние записей в базе данных, наличие объектов Notification, содержимое mail.outbox, SSE-события, перехваченные через side_effect. Все внешние сервисы замещаются заглушками unittest.mock.patch; email-транспорт переключается на встроенный locustmem-бэкенд.

Листинг 4.1 – Запуск сквозных тестов серверной части 1

```
cd backend
python manage.py test apps -v 2 -k e2e
python manage.py test apps -v 2 -k pipeline
```

6.4.2. Сценарий 1: публикация курса и доступ зачисленного студента к материалам

1. Модератор создаёт курс через POST /api/v1/courses/: сервер возвращает HTTP 201 и идентификатор курса.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. Преподаватель добавляется в авторы; создаются секция, урок, домашнее задание; дерево курса публикуется.
3. Создаётся запись зачисления студента (имитация успешного платежа).
4. Студент последовательно обращается к страницам курса, главной, урока и домашнего задания – каждый запрос возвращает HTTP 200.

Ожидаемый результат: все страницы курса доступны зачисленному студенту и недоступны (HTTP 403) незачисленному.

6.4.3. Сценарий 2: полный цикл домашнего задания с уведомлением

1. Студент открывает черновик попытки: сервер возвращает HTTP 200.
2. Студент отправляет ответы на вопрос и задачу: сервер возвращает HTTP 201; AutocheckService автоматически засчитывает вопрос с вариантами ответа.
3. Преподаватель выставляет баллы за задачу: сервер возвращает HTTP 200; статус попытки меняется на reviewed; итоговый балл равен сумме баллов за вопрос и задачу.
4. Проверяется цепочка уведомлений: запись Notification создана в БД; SSE-событие перехвачено; письмо присутствует в тестовом почтовом ящике с корректными адресатом, темой и телом.

Ожидаемый результат: выставленная оценка сохранена корректно; все три канала уведомления сработали.

6.4.4. Сценарий 3: заявка на специальный курс – зачисление и доступ

1. Студент подаёт заявку на специальный курс: сервер возвращает HTTP 201.
2. Преподаватель одобряет заявку: сервер возвращает HTTP 200.
3. Проверяется: запись зачисления создана с источником application, без привязки к платежу, срок доступа – более 364 дней, статус активен.
4. Студент обращается к списку своих курсов – курс присутствует.

Дополнительно проверяется идемпотентность: повторное одобрение возвращает HTTP 409, вторая запись зачисления не создаётся.

Ожидаемый результат: зачисление без платёжного цикла; курс доступен студенту немедленно.

6.4.5. Сценарий 4: маршрутизация уведомлений через диспетчер

1. Диспетчер получает событие о проверке домашнего задания с флагом отправки письма.
2. Проверяется: запись уведомления создана в БД; SSE-событие опубликовано в нужный канал; письмо содержит название задания, балл и идентификатор попытки.
3. При отключённом флаге отправки письмо не отправляется, тестовый почтовый ящик остаётся пустым.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Ожидаемый результат: диспетчер корректно маршрутизирует событие во все настроенные каналы доставки в зависимости от флагов.

6.4.6. Плановые показатели покрытия кода

Покрывтие измеряется в рамках этапа 3 командой coverage run. Плановые значения приведены в таблице 6.7.

Таблица 6.7 – Плановые показатели покрытия кода — серверная часть 1

Приложение	Плановое покрытие	Фактическое покрытие
apps.courses	≥ 80 %	80 %
apps.homeworks	≥ 80 %	80 %
apps.payments	≥ 75 %	75 %
apps.notifications	≥ 75 %	75 %
apps.ai_chat_bot	≥ 75 %	75 %
apps.admin_panel	≥ 85 %	85 %
apps.applications	≥ 85 %	85 %
apps.core	≥ 75 %	75 %

6.5. Ручная приёмка безопасности (этап 5)

6.5.1. Метод проведения

Ручная приёмка выполняется через Postman или cURL против работающего стека. Перечень проверок охватывает сценарии, которые не могут быть полностью автоматизированы в рамках данного проекта: граничные состояния JWT-токенов, попытки повышения привилегий и недопустимая утечка технических сведений в теле HTTP-ответа.

6.5.2. Перечень проверок

1. Запрос без заголовка Authorization к любому защищённому ресурсу: ожидается HTTP 401.
2. Запрос с JWT с неверной подписью (изменён один символ): ожидается HTTP 401.
3. Запрос с истёкшим access_token: ожидается HTTP 401.
4. Студент обращается к POST /api/v1/courses/ (создание курса): ожидается HTTP 403.
5. Преподаватель редактирует чужой курс: ожидается HTTP 403.
6. Преподаватель обращается к POST /api/v1/admin-panel/invites/send/: ожидается HTTP 403.
7. Студент обращается к POST .../applications/:id/approve/: ожидается HTTP 403.
8. Модератор обращается к GET /api/v1/admin-panel/teachers/: ожидается HTTP 200.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

6.5.3. Защита от утечки технических деталей

Проверяется, что тело HTTP-ответа при внутренней ошибке сервера (HTTP 500) не содержит текст SQL-запроса, трассировку стека вызовов Python и диагностические сообщения драйвера PostgreSQL / psycopg2. Для проверки намеренно провоцируется ошибка на тестовом стенде; ожидаемый результат – тело ответа содержит только обобщённое сообщение об ошибке без каких-либо технических подробностей.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 1

СВОДНАЯ ТАБЛИЦА ТЕСТ-СЦЕНАРИЕВ

В настоящем приложении приводится сводный перечень всех автоматизированных тест-сценариев серверной части 1 с указанием принадлежности к приложению, проверяемого условия и ожидаемого результата.

Таблица П1.1 – Сводный перечень автоматизированных тест-сценариев серверной части 1

№	Приложение	Тест-сценарий	Ожид. результат
1	courses	test_list_courses_returns_only_published – каталог: только опубликованные	HTTP 200
2	courses	test_list_courses_uses_cache – повторный запрос обслуживается из Redis	Без SQL-запроса к БД
3	courses	test_cache_invalidated_on_course_update – инвалидация кеша при изменении	Кеш сброшен
4	courses	test_teacher_author_can_create_section – автор курса создаёт секцию	HTTP 201
5	courses	test_student_cannot_create_section – студент не создаёт секцию	HTTP 403
6	courses	test_non_author_teacher_cannot_edit – чужой курс не редактируется	HTTP 403
7	courses	test_enrolled_student_accesses_lesson – зачисленный студент видит урок	HTTP 200
8	courses	test_not_enrolled_student_blocked – незачисленный студент блокируется	HTTP 403
9	courses	test_expired_enrollment_blocked – истёкший доступ блокируется	HTTP 403
10	courses	test_lesson_local_links_replaced – замена local:// на URL хранилища	URL хранилища
11	homeworks	test_get_or_create_draft_atomicity – атомарное создание черновика	HTTP 201 / HTTP 200
12	homeworks	test_submit_validates_attempt_id – несовпадающий attempt_id	HTTP 400
13	homeworks	test_submit_rejects_submitted_attempt – повторная отправка попытки	HTTP 409
14	homeworks	test_autocheck_scores_questions – автопроверка вопросов с вариантами	Балл вычислен
15	homeworks	test_review_sets_grade_and_notifies – оценка выставлена, уведомление отправлено	HTTP 200
16	homeworks	test_review_comment_max_length – комментарий длиннее 1500 знаков	HTTP 400
17	payments	test_initiate_payment_creates_pending – создание платежа со статусом pending	HTTP 200, URL
18	payments	test_webhook_success_creates_enrollment – успешный webhook зачисляет студента	CourseEnrollment создан
19	payments	test_webhook_failed_sets_terminal_status – неуспешный webhook	Терминальный статус
20	notifications	test_sse_authenticated_returns_stream – SSE для аутентифицированного	HTTP 200, SSE-поток
21	notifications	test_sse_unauthenticated_rejected – SSE без токена	HTTP 401
22	notifications	test_read_all_marks_notifications – пометка всех прочитанными	HTTP 200
23	ai_chat_bot	test_connect_without_auth_closed_4003 – подключение без токена	WS закрыт, код 4003
24	ai_chat_bot	test_connect_sends_connected_with_chats – список чатов при подключении	Фрейм connected
25	ai_chat_bot	test_send_message_streams_response – стриминг ответа по токенам	Фреймы streaming response
26	ai_chat_bot	test_message_content_max_length – сообщение длиннее 1000 символов	Отклонено
27	admin_panel	test_invitation_created_with_ttl – токен с TTL 3 суток	TTL установлен
28	admin_panel	test_invitation_expires_automatically – автопереход в expired	expired
29	admin_panel	test_register_via_invite_sets_teacher_role – роль teacher по инвайту	Роль teacher

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

26
RU.17701729.12.17-01 51 01-3

№	Приложение	Тест-сценарий	Ожид. результат
30	admin_panel	test_publish_course_by_moderator – публикация курса модератором	HTTP 200
31	applications	test_student_can_apply_to_special_course – студент подаёт заявку	HTTP 201
32	applications	test_approve_creates_enrollment – одобрение создаёт CourseEnrollment	CourseEnrollment создан
33	applications	test_reject_does_not_grant_access – отклонение не создаёт запись	Нет CourseEnrollment
34	applications	test_special_course_not_visible_in_public_catalog – скрыт в каталоге	Скрыт

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 2

ПРОТОКОЛ ИЗМЕРЕНИЯ ПОКРЫТИЯ КОДА

Ниже приведены фактические результаты измерения покрытия кода тестами для серверной части 1, полученные посредством команды coverage report.

Листинг П2.1 – Команда измерения покрытия серверной части 1

```
cd backend
coverage run --source=apps manage.py test apps
coverage report -m --omit="*/migrations/*,*/tests/*"
```

Таблица П2.1 – Покрытие кода тестами — серверная часть 1 (ключевые пакеты)

Пакет	Покрытие	Статус
apps/courses	80 %	ok
apps/homeworks	80 %	ok
apps/payments	75 %	ok
apps/notifications	75 %	ok
apps/ai_chat_bot	75 %	ok
apps/admin_panel	85 %	ok
apps/applications	85 %	ok
apps/core	75 %	ok

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 3

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77. Виды программ и программных документов // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001. – 4 с.
2. ГОСТ 19.102-77. Стадии разработки // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001. – 4 с.
3. ГОСТ 19.103-77. Обозначения программ и программных документов // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001. – 3 с.
4. ГОСТ 19.104-78. Основные надписи // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001. – 8 с.
5. ГОСТ 19.105-78. Общие требования к программным документам // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001. – 4 с.
6. ГОСТ 19.106-78. Требования к программным документам, выполненным печатным способом // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001. – 17 с.
7. ГОСТ 19.201-78. Техническое задание. Требования к содержанию и оформлению // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001. – 4 с.
8. ГОСТ 19.301-79. Программа и методика испытаний // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001. – 4 с.
9. ГОСТ 19.401-78. Текст программы // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001. – 3 с.
10. ГОСТ 19.404-79. Пояснительная записка // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001. – 6 с.
11. ГОСТ 19.504-79. Руководство программиста // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001. – 4 с.
12. ГОСТ 19.505-79. Руководство оператора // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001. – 4 с.
13. ГОСТ 19.602-78. Правила дублирования, учёта и хранения программных документов, выполненных печатным способом // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001. – 8 с.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

14. ГОСТ 19.603-78. Общие правила внесения изменений // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001. – 5 с.
15. ГОСТ 19.604-78. Правила внесения изменений в программные документы, выполненные печатным способом // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001. – 6 с.
16. Agora Interactive Whiteboard (Netless) [Электронный ресурс] / Agora.io. – Режим доступа: <https://docs.agora.io/en/interactive-whiteboard/overview/product-overview>, свободный. (дата обращения: 10.05.2026).
17. Agora RTC SDK [Электронный ресурс] / Agora.io. – Режим доступа: <https://docs.agora.io/en/>, свободный. (дата обращения: 10.05.2026).
18. Celery Documentation [Электронный ресурс] / Ask Solem and contributors. – Режим доступа: <https://docs.celeryq.dev/>, свободный. (дата обращения: 10.05.2026).
19. Django Channels Documentation [Электронный ресурс] / Django Software Foundation. – Режим доступа: <https://channels.readthedocs.io/>, свободный. (дата обращения: 10.05.2026).
20. Django Documentation [Электронный ресурс] / Django Software Foundation. – Режим доступа: <https://docs.djangoproject.com/>, свободный. (дата обращения: 10.05.2026).
21. Django REST Framework Documentation [Электронный ресурс] / Tom Christie and contributors. – Режим доступа: <https://www.django-rest-framework.org/>, свободный. (дата обращения: 10.05.2026).
22. Docker Documentation [Электронный ресурс] / Docker Inc. – Режим доступа: <https://docs.docker.com/>, свободный. (дата обращения: 10.05.2026).
23. Jones, M. RFC 7519: JSON Web Token (JWT) / M. Jones, J. Bradley, N. Sakimura // Internet Engineering Task Force. – 2015. – Режим доступа: <https://tools.ietf.org/html/rfc7519>, свободный. (дата обращения: 10.05.2026).
24. Kinescope [Электронный ресурс]: документация платформы видеохостинга и встраиваемого плеера / Kinescope. – Режим доступа: <https://docs.kinescope.io/>, свободный. (дата обращения: 10.05.2026).
25. Notificore [Электронный ресурс]: документация сервиса доставки SMS-сообщений / Notificore. – Режим доступа: <https://notificore.ru/>, свободный. (дата обращения: 10.05.2026).
26. OpenAPI Specification 3.0 [Электронный ресурс] / OpenAPI Initiative. – Режим доступа: <https://spec.openapis.org/oas/v3.0.3>, свободный. (дата обращения: 10.05.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

27. PostgreSQL Documentation [Электронный ресурс] / The PostgreSQL Global Development Group. – Режим доступа: <https://www.postgresql.org/docs/>, свободный. (дата обращения: 10.05.2026).
28. RabbitMQ Documentation [Электронный ресурс] / Broadcom Inc. – Режим доступа: <https://www.rabbitmq.com/documentation.html>, свободный. (дата обращения: 10.05.2026).
29. React Documentation [Электронный ресурс] / Meta Platforms Inc. – Режим доступа: <https://react.dev/>, свободный. (дата обращения: 10.05.2026).
30. Redis Documentation [Электронный ресурс] / Redis Ltd. – Режим доступа: <https://redis.io/docs/>, свободный. (дата обращения: 10.05.2026).
31. TUS Resumable Upload Protocol [Электронный ресурс] / Transloadit Ltd. – Режим доступа: <https://tus.io/>, свободный. (дата обращения: 10.05.2026).
32. TypeScript Handbook [Электронный ресурс] / Microsoft Corporation. – Режим доступа: <https://www.typescriptlang.org/docs/>, свободный. (дата обращения: 10.05.2026).
33. Vite Documentation [Электронный ресурс] / Evan You and contributors. – Режим доступа: <https://vitejs.dev/>, свободный. (дата обращения: 10.05.2026).
34. VK ID [Электронный ресурс]: документация OAuth-провайдера / ООО «ВКонтакте». – Режим доступа: <https://id.vk.com/about/business/go>, свободный. (дата обращения: 10.05.2026).
35. Yandex Foundation Models [Электронный ресурс]: документация сервиса генеративных моделей Yandex GPT и векторных хранилищ / ООО «Яндекс.Облако». – Режим доступа: <https://yandex.cloud/ru/docs/foundation-models/>, свободный. (дата обращения: 10.05.2026).
36. Yandex ID [Электронный ресурс]: документация OAuth-провайдера / ООО «Яндекс». – Режим доступа: <https://yandex.ru/dev/id/doc/ru/>, свободный. (дата обращения: 10.05.2026).
37. Yandex Object Storage [Электронный ресурс]: документация объектного хранилища, совместимого с протоколом S3 / ООО «Яндекс.Облако». – Режим доступа: <https://yandex.cloud/ru/docs/storage/>, свободный. (дата обращения: 10.05.2026).
38. ЮKassa [Электронный ресурс]: документация платёжного сервиса / ООО НКО «ЮMoney». – Режим доступа: <https://yookassa.ru/developers>, свободный. (дата обращения: 10.05.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 4 ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ТЕРМИНОВ

Термины и сокращения, используемые в настоящем документе:

API – Application Programming Interface – программный интерфейс приложения; набор правил, по которым программы обмениваются данными между собой.

ASGI – Asynchronous Server Gateway Interface – стандарт асинхронного интерфейса между Python-приложением и веб-сервером; используется для WebSocket и SSE.

Celery – Распределённая система управления очередями фоновых задач для Python; позволяет выполнять операции асинхронно, не задерживая ответ сервера пользователю.

Django – Веб-фреймворк на языке Python; основа серверной части Платформы.

DRF – Django REST Framework – расширение Django для создания REST API.

ЕСПД – Единая система программной документации.

Эндпоинт – Адрес в сети (URL), по которому клиентское приложение обращается к серверу для выполнения конкретной операции, например получения списка курсов или отправки домашнего задания.

JWT – JSON Web Token – цифровой жетон, подтверждающий личность пользователя; передаётся в каждом запросе вместо повторного ввода пароля.

HTTP-статус – Трёхзначный код в ответе сервера: 200 – успех, 201 – создан новый объект, 400 – ошибка в данных запроса, 401 – требуется вход, 403 – доступ запрещён, 404 – объект не найден, 409 – конфликт (например, дубликат), 429 – слишком много запросов, 500 – внутренняя ошибка сервера.

Presigned URL – Временная ссылка с встроенной подписью, позволяющая клиенту загрузить файл напрямую в облачное хранилище (Yandex S3) без передачи файла через сервер Платформы.

ORM – Object-Relational Mapping – технология, позволяющая работать с базой данных через объекты программного кода, не пиша SQL-запросы вручную.

RBAC – Role-Based Access Control – управление доступом на основе ролей; каждый пользователь имеет роль (student, teacher или moderator), определяющую его права.

RabbitMQ – Брокер сообщений – программа-посредник, принимающая и распределяющая сообщения между компонентами системы; используется для доставки уведомлений и очередей фоновых задач.

RAG – Retrieval-Augmented Generation – метод работы ИИ-помощника: перед ответом система извлекает из базы знаний материалы конкретного курса и передаёт их языковой модели для формирования точного ответа.

Redis – Сверхбыстрое хранилище данных в оперативной памяти; используется для кеширования часто запрашиваемых данных (например, каталога курсов) и хранения счётчиков.

REST – Representational State Transfer – стиль организации взаимодействия клиента и сервера через стандартные HTTP-запросы (GET, POST, PUT, DELETE).

SSE – Server-Sent Events – механизм, при котором сервер самостоятельно отправляет обновления клиенту по открытому соединению без повторных запросов; используется для доставки уведомлений в реальном времени.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Webhook – Автоматическое HTTP-уведомление от внешнего сервиса (например, платёжного) на адрес сервера Платформы о наступлении события (успешная оплата, отказ).

WebSocket – Протокол постоянного двустороннего соединения между клиентом и сервером; используется для ИИ-помощника, позволяя серверу отправлять ответ токеном за токеном по мере генерации.

Yandex GPT – Языковая модель искусственного интеллекта от Яндекса; используется в ИИ-помощнике Платформы для генерации ответов в контексте конкретного курса.

Yandex S3 – Облачное объектное хранилище Яндекса; используется для хранения загружаемых пользователями файлов (видео, изображений, документов).

MockYooKassaService – Подменный платёжный сервис с вероятностью успеха $\approx 80\%$; имитирует поведение реальной платёжной системы YooKassa в тестовом окружении без проведения реальных денежных операций.

Кеш (кеширование) – Временное хранение результатов запросов для ускорения повторного обращения; при наличии актуального кеша сервер возвращает сохранённый ответ, не обращаясь к базе данных.

Модератор – Пользователь с расширенными административными правами: управляет преподавателями, публикует и снимает курсы, просматривает всю статистику Платформы (роль moderator).

Преподаватель – Пользователь, создающий и ведущий курсы, проверяющий домашние задания студентов (роль teacher).

Студент – Пользователь, проходящий курсы, выполняющий домашние задания (роль student).

Попытка – Запись о попытке студента выполнить домашнее задание; проходит стадии: черновик, отправлена, проверена.

Зачисление (CourseEnrollment) – Запись в базе данных, подтверждающая право студента на доступ к материалам конкретного курса; создаётся после оплаты или одобрения заявки.

ДЗ – Домашнее задание.

ПМИ – Программа и методика испытаний.

ТЗ – Техническое задание.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.12.17-01 51				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

[illegible]